Java Programming Assignment & Code Solutions

Comprehensive Solutions Covering JALA Academy Java Curriculum (Basics to Collections)

1. Java Basics

1.1 & 1.2 Class, Object, Method & Print Name

Write a program to print your name and understand basic structure.

```
public class PrintName {
    public static void printMyName() {
        System.out.println("Student Name: John Doe");
    }
    public static void main(String[] args) {
        printMyName();
    }
}
```

1.4 Data Types Demonstration

Define variables for int, boolean, char, float, double and print.

```
public class DataTypesDemo {
    public static void main(String[] args) {
        int age = 25;
        boolean isJavaFun = true;
        char grade = 'A';
        float temperature = 98.6f;
        double salary = 75000.50;

        System.out.println("Int: " + age);
        System.out.println("Boolean: " + isJavaFun);
        System.out.println("Char: " + grade);
        System.out.println("Float: " + temperature);
        System.out.println("Double: " + salary);
    }
}
```

1.5 Local vs Global Variables

Define local and global variables with the same name and understand scope.

```
public class VariableScope {
    int value = 100; // Global / Instance variable

    void display() {
        int value = 50; // Local variable
        System.out.println("Local value: " + value);
        System.out.println("Global value: " + this.value);
    }

    public static void main(String[] args) {
        new VariableScope().display();
    }
}
```

2. Operators

2.1 Arithmetic & Relational Operators

*Write functions for arithmetic operators (+, -, *, /) and relational operators.*

```
public class OperatorsDemo {
    public static void main(String[] args) {
        int a = 20, b = 10;
        // Arithmetic
        System.out.println("Addition: " + (a + b));
        System.out.println("Subtraction: " + (a - b));
        System.out.println("Multiplication: " + (a * b));
        System.out.println("Division: " + (a / b));

        // Relational
        System.out.println("a > b: " + (a > b));
        System.out.println("a == b: " + (a == b));

        // Logical
        boolean x = true, y = false;
        System.out.println("x && y: " + (x && y));
        System.out.println("x || y: " + (x || y));
    }
}
```

3. Loops

3.1, 3.2, 3.6 & 3.7 Loops (For, While, Do-While)

Implement various loops including printing 1 to 20 and even numbers.

```
public class LoopsDemo {
    public static void main(String[] args) {
        // For Loop
        for (int i = 1; i <= 10; i++) {
            System.out.print("Bright IT Career ");
        }
        System.out.println();

        // While Loop (1 to 20)
        int j = 1;
        while (j <= 20) {
            System.out.print(j + " ");
            j++;
        }
        System.out.println();

        // Even numbers between 10 and 100 using while
        int k = 10;
        while (k <= 100) {
            if (k % 2 == 0) System.out.print(k + " ");
            k += 2;
        }
    }
}
```

4. Arrays

4.1, 4.2, 4.8 & 4.9 Array Operations

Add integer values, find average, min/max, and reverse array.

```
public class ArrayOperations {
    public static void main(String[] args) {
        int[] arr = {15, 8, 25, 42, 10};
        int sum = 0;
        int min = arr[0], max = arr[0];

        for (int n : arr) {
            sum += n;
            if (n < min) min = n;
            if (n > max) max = n;
        }
        double avg = (double) sum / arr.length;

        System.out.println("Sum: " + sum);
        System.out.println("Average: " + avg);
        System.out.println("Min: " + min + ", Max: " + max);
    }
}
```

5. Static

5.1 Static and Instance Members

Write a class with static and instance variables and methods.

```
public class StaticTest {
    static int sVar1 = 10, sVar2 = 20;
    int iVar1 = 30, iVar2 = 40;

    static void sMethod1() {
        System.out.println("Static method 1. sVar1: " + sVar1);
    }
    static void sMethod2() {
        sMethod1();
    }

    void iMethod1() {
        System.out.println("Instance method 1. Accessing static sVar1: " + sVar1);
        System.out.println("Instance variable iVar1: " + iVar1);
    }

    public static void main(String[] args) {
        sMethod1();
        StaticTest obj = new StaticTest();
        obj.iMethod1();
        System.out.println("Main method printing static: " + sVar1 + ", instance: " + obj.iVar1);
    }
}
```

6. Strings

6.1 to 6.14 String Operations

Concatenation, length, substring, indexOf, comparison, and conversion.

```
public class StringsDemo {
    public static void main(String[] args) {
        String s1 = "Hello";
        String s2 = "World";

        // Concatenation & Length
        String s3 = s1 + " " + s2;
        System.out.println("Concat: " + s3 + ", Length: " + s3.length());

        // Substring & indexOf
        System.out.println("Substring(6): " + s3.substring(6));
        System.out.println("Index of 'W': " + s3.indexOf('W'));

        // Comparison
        System.out.println("Equals: " + s1.equals("Hello"));
        System.out.println("CompareTo: " + s1.compareTo("World"));

        // Uppercase & Lowercase
        System.out.println("Upper: " + s3.toUpperCase());
    }
}
```

7. Inheritance

7.1 Inheritance & Method Overriding

Classes A, B, C with specific and overridden methods.

```
class A {
    void method1() { System.out.println("A: method1"); }
    void method2() { System.out.println("A: method2"); }
    void show() { System.out.println("A: show"); }
}
class B extends A {
    void method3() { System.out.println("B: method3"); }
    void method4() { System.out.println("B: method4"); }
    @Override void show() { System.out.println("B: show"); }
}
class C extends B {
    void method5() { System.out.println("C: method5"); }
    void method6() { System.out.println("C: method6"); }
    @Override void show() { System.out.println("C: show"); }
}
public class TestInheritance {
    public static void main(String[] args) {
        A objA = new A(); objA.show();
        B objB = new B(); objB.show();
        A ref = new C(); ref.show(); // Runtime polymorphism
    }
}
```

8. Access Modifiers

8.1 to 8.4 Access Modifiers Demonstration

Private, Default, Protected, and Public access levels.

```
class AccessDemo {
    private int privVar = 10;
    int defVar = 20; // Default
    protected int protVar = 30;
    public int pubVar = 40;

    private void privMethod() {
        System.out.println("Private method. privVar: " + privVar);
    }
    public void accessPrivate() {
        privMethod(); // Accessible within class
    }
}
public class TestAccess {
    public static void main(String[] args) {
        AccessDemo obj = new AccessDemo();
        // System.out.println(obj.privVar); // Compile error
        System.out.println("Default: " + obj.defVar);
        System.out.println("Protected: " + obj.protVar);
        System.out.println("Public: " + obj.pubVar);
        obj.accessPrivate();
    }
}
```

9. Abstract Class

9.1 to 9.4 Abstract Class Implementation

Abstract class with abstract and non-abstract methods.

```
abstract class Animal {
    abstract void sound(); // Abstract method
    void sleep() {         // Non-abstract method
        System.out.println("Animal is sleeping...");
    }
}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks: Woof Woof");
    }
}
public class TestAbstract {
    public static void main(String[] args) {
        Animal myDog = new Dog();
        myDog.sound();
        myDog.sleep();
    }
}
```

10. Interfaces

10.1 & 10.4 Interfaces Implementation

Creating interfaces and implementing them in classes.

```
interface Flyable { void fly(); }
interface Swimmable { void swim(); }

class Duck implements Flyable, Swimmable {
    public void fly() { System.out.println("Duck flies low."); }
    public void swim() { System.out.println("Duck swims in water."); }
}

public class TestInterface {
    public static void main(String[] args) {
        Duck d = new Duck();
        d.fly();
        d.swim();
    }
}
```

11. This and Super

11.1 & 11.2 Using this and super

Accessing current class and parent class members.

```
class Parent {
    String name = "Parent Class";
}
class Child extends Parent {
    String name = "Child Class";
    void display() {
        System.out.println("Using this: " + this.name);
        System.out.println("Using super: " + super.name);
    }
}
public class TestThisSuper {
    public static void main(String[] args) {
        new Child().display();
    }
}
```

12. Constructors

12.1 Constructors Overloading

Default, 1-argument, and 2-argument constructors.

```
class Student {
    String name;
    int id;

    Student() { System.out.println("Default Constructor"); }
    Student(String name) { this.name = name; System.out.println("1-Arg: " + name); }
    Student(String name, int id) {
        this.name = name; this.id = id;
        System.out.println("2-Arg: " + name + ", " + id);
    }
}
public class TestConstructors {
    public static void main(String[] args) {
        new Student();
        new Student("Alice");
        new Student("Bob", 101);
    }
}
```

13. Method Overloading

13.1 & 13.2 Method Overloading Examples

Methods with same name but different parameters.

```
public class MathOperations {
    int add(int a, int b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
    double add(double a, double b) { return a + b; }

    public static void main(String[] args) {
        MathOperations mo = new MathOperations();
        System.out.println("Sum 2 ints: " + mo.add(5, 10));
        System.out.println("Sum 3 ints: " + mo.add(5, 10, 15));
        System.out.println("Sum 2 doubles: " + mo.add(5.5, 10.2));
    }
}
```

14. Exceptions

14.1, 14.2 & 14.7 Exception Handling

Try-catch, arithmetic exception, and finally block.

```
public class ExceptionDemo {
    public static void main(String[] args) {
        try {
            int data = 50 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Caught ArithmeticException: " + e.getMessage());
        } finally {
            System.out.println("Finally block executed always.");
        }
    }
}
```

15. Java IO

15.5 & 15.6 FileReader and FileWriter

Writing and reading text files using FileReader and FileWriter.

```
import java.io.*;

public class FileIODemo {
    public static void main(String[] args) {
        try {
            // Write
            FileWriter writer = new FileWriter("test.txt");
            writer.write("Hello JALA Academy!");
            writer.close();

            // Read
            FileReader reader = new FileReader("test.txt");
            int ch;
            System.out.print("File Content: ");
            while ((ch = reader.read()) != -1) {
                System.out.print((char) ch);
            }
            reader.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

16. Collections

16.1 & 16.2 ArrayList and HashMap Operations

Using ArrayList and HashMap collections in Java.

```
import java.util.*;

public class CollectionsDemo {
    public static void main(String[] args) {
        // ArrayList
        List list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");
        System.out.println("ArrayList: " + list);

        // Iterator
        Iterator it = list.iterator();
        System.out.print("Iterating: ");
        while(it.hasNext()) {
            System.out.print(it.next() + " ");
        }
        System.out.println();

        // HashMap
        Map map = new HashMap<>();
        map.put(1, "Alice");
        map.put(2, "Bob");
        map.put(3, "Charlie");
        System.out.println("HashMap: " + map);
        System.out.println("Student ID 2: " + map.get(2));
    }
}
```